

# DICE Network

Distributed Infrastructure for Cryptographic Entropy

---

## Technology Stack & Operational Security

From Firmware to Deployment

CONFIDENTIAL — Internal Use Only

# PART 1: TECHNOLOGY STACK

---

## 1. Firmware (ESP32-S3)

The firmware layer runs on ESP32-S3 microcontrollers and forms the foundation of the DICE network. All firmware is written in C using the Espressif ESP-IDF v5.x framework. This is not negotiable — Arduino, MicroPython, and Lua do not provide the deterministic behavior and low-level control required for a security-critical system.

### 1.1 Core Framework

**Framework:** ESP-IDF v5.x (Espressif IoT Development Framework)

**Language:** C (not C++, not Arduino)

**Build System:** idf.py with GCC-based toolchain

**RTOS:** FreeRTOS (included in ESP-IDF)

### 1.2 Key Libraries & APIs

- `esp_tls` — mTLS WebSocket connections to the coordinator
- `mbedtls` — ECDSA signing (`secp256k1`) for entropy submissions
- `nvs_flash` — Encrypted NVS storage for device credentials
- `esp_secure_boot` — Secure boot v2 (eFuse-based, irreversible)
- `esp_flash_encryption` — Flash encryption (eFuse-based, irreversible)
- `esp_websocket_client` — Persistent WebSocket connection to coordinator
- `esp_random()` — Hardware RNG API for entropy generation

### 1.3 Cryptographic Identity

Each device uses ECDSA with `secp256k1` for its keypair. This aligns with Solana's key format, allowing the same keypair to serve both as the device's network identity and its on-chain identity. The `micro-ecc` library or `mbedtls`'s built-in ECC implementation handles elliptic curve operations on the ESP32.

### 1.4 Firmware Design: Immutable After Production

Once firmware is flashed and eFuses are burned during manufacturing, no firmware updates will ever be deployed. This is an intentional security decision — immutable firmware means no remote exploit can persist a modified image. The tradeoff is that any bugs or certificate issues post-production require physical device replacement.

## 2. Transport Layer

### 2.1 Protocol: WebSocket over TLS (wss://)

Nodes maintain a persistent bidirectional connection to the coordinator using WebSocket over TLS. Plain WebSocket (ws://) is never used — all data in transit is encrypted.

WebSocket is chosen over raw TCP or HTTP polling because the system requires bidirectional real-time communication: job assignments flow from coordinator to node, while heartbeats and entropy results flow from node to coordinator.

### 2.2 Mutual TLS (mTLS)

Both sides of the connection authenticate each other during the TLS handshake. The coordinator presents its server certificate, and the ESP32 node presents its device certificate. Both verify against the shared private Certificate Authority.

This provides encrypted transport and authentication in a single layer. The coordinator knows it is communicating with a registered device, and the device knows it is talking to the legitimate coordinator.

### 2.3 Security Properties

- Confidentiality — TLS encrypts all data in transit
- Authentication — mTLS verifies both sides of the connection
- Integrity — TLS detects tampering; individual entropy payloads are also signed with device keys
- Replay protection — TLS session management prevents replay attacks at the transport layer
- Connection hijacking resistance — mTLS re-authenticates on every reconnection
- Denial of service mitigation — mTLS handshake must complete before any resources are allocated

### 2.4 Implementation

**ESP32 side:** esp\_tls with mbedTLS (native to ESP-IDF)

**Coordinator side:** rustls (memory-safe TLS implementation, not OpenSSL)

**Protocol library:** tokio-tungstenite (Rust WebSocket over TLS)

## 3. Coordinator Server

### 3.1 Language: Rust

The coordinator is written in Rust. For a security-critical real-time system, Rust provides memory safety without garbage collection pauses, strong concurrency primitives, and native compatibility with the Solana ecosystem.

### 3.2 Core Dependencies

- Tokio — Async runtime for concurrent connection handling
- tokio-tungstenite — WebSocket server with TLS support
- rustls — Memory-safe TLS backend (replaces OpenSSL)
- sqlx — Async database driver for PostgreSQL
- solana-sdk / anchor-client — On-chain interaction

### 3.3 Database: PostgreSQL

PostgreSQL stores the node registry (public keys, latency measurements, uptime, job counts), active randomness request state, commit-reveal round data, and audit logs. The database runs locally on the coordinator VPS initially and can be migrated to a managed instance at scale.

### 3.4 Coordinator Responsibilities

- Accept and verify mTLS WebSocket connections from nodes
- Maintain real-time node registry (latency, uptime, job count)
- Subscribe to Solana for incoming randomness requests
- Execute node selection algorithm per request
- Manage commit-reveal rounds with timeouts
- Submit finalized randomness results on-chain
- Distribute rewards to participating nodes

## 4. Solana Smart Contract

### 4.1 Framework: Anchor

The on-chain program is written in Rust using the Anchor framework. Anchor provides structured account management, automatic serialization/deserialization, and cleaner instruction handlers compared to raw Solana programs. The program compiles to BPF bytecode for deployment.

### 4.2 Program Accounts

- DeviceRegistry — Stores registered node public keys
- RandomnessRequest — Stores request ID, status, selected nodes
- CommitRecord — Stores commit hashes per node per request
- RevealRecord — Stores revealed entropy values
- RandomnessResult — Final combined randomness output

### 4.3 Program Instructions

- register\_device — Register a new node's public key
- request\_randomness — Developer contract requests randomness
- submit\_commit — Node submits hash(entropy)
- submit\_reveal — Node reveals entropy with signature
- finalize\_randomness — Combine entropy and store result
- claim\_rewards — Node claims earned rewards

### 4.4 Testing

- solana-program-test — Local program simulation
- Bankrun — Fast integration testing
- Anchor CLI (anchor test) — Runs local validator with TypeScript integration tests
- Trident — Fuzz testing for the program

## 5. Infrastructure & Deployment

### 5.1 Server

**Platform:** VPS (Hetzner, OVH, or similar — avoid hyperscaler lock-in)

**OS:** Hardened Debian or Ubuntu Server

**Containerization:** Docker with minimal base image (distroless or Alpine)

**Reverse Proxy:** Caddy or nginx for public API TLS termination (node-facing mTLS handled directly by the Rust process)

### 5.2 Monitoring

**Metrics:** Prometheus + Grafana for node health, coordinator performance, job completion rates

**Alerting:** Prometheus alerting rules for node disconnections, coordinator crashes, Solana transaction failures, failed mTLS handshakes

**Audit Logging:** All coordinator actions logged and shipped to a separate server for tamper-resistant storage

### 5.3 Solana RPC

**Provider:** Helius or Triton for mainnet RPC access

**Devnet:** Public Solana devnet RPC for testing

### 5.4 Development & CI/CD Tools

- Firmware: ESP-IDF toolchain with idf.py; QEMU for ESP32-S3 emulation in CI; physical test bench with 3–5 devices
- Coordinator: Cargo toolchain; cargo test; docker-compose for local full-stack testing (coordinator + Postgres + solana-test-validator)
- Smart Contract: Anchor CLI; anchor test; Trident fuzz testing
- End-to-End: Staging environment with 5 physical nodes hitting a devnet coordinator connected to Solana devnet

### 5.5 Technology Stack Summary

| Layer          | Technology                                        |
|----------------|---------------------------------------------------|
| Firmware       | ESP-IDF v5.x, C, mbedTLS, FreeRTOS                |
| Device Crypto  | ECDSA secp256k1, Secure Boot v2, Flash Encryption |
| Transport      | WebSocket over mTLS (rustls)                      |
| Coordinator    | Rust, Tokio, tokio-tungstenite, PostgreSQL        |
| Smart Contract | Rust, Anchor Framework, Solana                    |
| PKI            | step-ca, Private Root + Intermediate CA           |

|                |                                                |
|----------------|------------------------------------------------|
| Provisioning   | esptool.py, esepfuse.py, Custom Python Scripts |
| Infrastructure | Docker, Debian VPS, Prometheus/Grafana         |
| RPC            | Helius or Triton                               |

## PART 2: OPERATIONAL SECURITY

---

### 6. Root CA Key Protection

The root CA private key is the single most critical secret in the entire DICE system. If compromised, an attacker can mint valid device certificates, impersonate nodes, and collapse the entire trust model. Protection of this key is treated with the highest priority.

#### 6.1 Generation: Air-Gapped Machine

The root CA keypair is generated on a dedicated air-gapped machine — a cheap laptop that has never connected to the internet. The WiFi card and Bluetooth module should be physically removed. A minimal Linux installation (Debian with no desktop environment) is installed from a USB drive, along with the step-ca CLI tools.

The root CA private key is never generated on a cloud server, a development machine, or any internet-connected device under any circumstances.

#### 6.2 Storage: Encrypted USB Drives

The root CA private key is encrypted with a strong passphrase and stored on two LUKS-encrypted USB drives. These drives are stored in two separate physical locations: a fireproof safe at a primary location and a bank safe deposit box or equivalent at a secondary location.

#### 6.3 Passphrase: Shamir's Secret Sharing

The USB drive passphrase is split using Shamir's Secret Sharing (ssss package on Linux) into 3 shares with a threshold of 2. No single person can reconstruct the passphrase alone.

- Share 1: Held by primary operator
- Share 2: Held by co-founder or trusted partner
- Share 3: Sealed envelope in a separate physical safe

#### 6.4 Usage: Extremely Rare

The root CA key is only used to sign intermediate CA certificates. This happens once during initial setup and approximately once every 5–10 years for rotation. Each use follows the same procedure: boot the air-gapped machine, connect the USB drive, sign the required certificate, copy the signed certificate to a separate clean USB, and secure everything again.

## 7. Intermediate CA Management

The intermediate CA handles day-to-day signing operations: device certificates during provisioning and coordinator certificates during deployment. While more exposed than the root CA by necessity, it still requires strong protection.

### 7.1 Location & Protection

The intermediate CA private key lives on the provisioning station's disk, protected by LUKS full disk encryption. The provisioning station is powered off when not actively provisioning devices.

### 7.2 Lifetime

The intermediate CA certificate has a validity period of 5–10 years. When it expires, a new intermediate is signed using the air-gapped root CA machine. This renewal must be planned well in advance.

### 7.3 Compromise Response

If the intermediate CA key is compromised, it is revoked using the root CA (sign a new intermediate, push the old one to a revocation list). The coordinator then rejects connections from devices whose certificates were signed by the revoked intermediate. Those devices are effectively retired. This is painful but is the only secure option given the no-firmware-update policy.

## 8. Certificate Infrastructure (Private PKI)

DICE uses a private PKI — not a public CA like Let's Encrypt. Both sides of every connection are controlled by DICE, so a private CA provides tighter control and eliminates external dependencies.

### 8.1 CA Software: step-ca (Smallstep)

step-ca is an open-source, lightweight certificate authority purpose-built for internal PKI.

### 8.2 PKI Structure

- Root CA: Kept offline, 20-year lifetime
- Intermediate CA: Signs device and coordinator certificates, 5–10 year lifetime
- Device Certificates: Signed by intermediate CA, embedded during manufacturing
- Coordinator Certificate: Signed by intermediate CA, deployed with the server

### 8.3 Certificate Lifetime Planning

Since devices cannot receive firmware updates, certificate lifetimes must be planned carefully. Device certificates should be long-lived (10+ years). Alternatively, a small NVS partition can be reserved for certificate rotation that does not require a full firmware update, but this introduces additional complexity that must be evaluated against the threat model.



## 9. Device Provisioning Station

The provisioning station is a dedicated physical machine that serves a single purpose: flashing and provisioning ESP32 devices. It is never used as a daily driver or development machine.

### 9.1 Hardware

- Small form-factor PC (Intel NUC or similar)
- No wireless capability (wired Ethernet only, connected only when needed for on-chain registration)
- BIOS password set
- USB boot disabled after OS installation
- Full disk encryption with LUKS

### 9.2 Software

- Minimal Debian or Ubuntu Server installation
- ESP-IDF toolchain
- Python 3 with esptool and esptool
- step-ca CLI for signing device certificates with the intermediate CA
- Custom provisioning script
- PostgreSQL client for logging to inventory database

### 9.3 Physical Security

The provisioning station is stored in a locked room or cabinet. When not in use for a provisioning batch, it is powered off and physically secured. Access is restricted to the primary operator and at most one other trusted person.

### 9.4 Provisioning Flow Per Device

1. Generate unique ECDSA secp256k1 keypair on the provisioning station (not on the device)
2. Flash the firmware binary
3. Write device private key and coordinator CA certificate to encrypted NVS
4. Enable Secure Boot v2 (burns eFuse — irreversible)
5. Enable Flash Encryption (burns eFuse — irreversible)
6. Record device public key in provisioning database
7. Register the public key on-chain via `register_device`

### 9.5 Device Key Generation Rationale

Device keypairs are generated on the provisioning station rather than on the ESP32 itself. At provisioning time, the device is in an unverified state (secure boot is not yet enabled), so a compromised device could theoretically generate a weak key. Generating on the provisioning station using `/dev/urandom` (backed by the kernel's CSPRNG) provides higher confidence in

key quality. The private key exists in the provisioning station's memory only during flashing and is securely wiped afterward. The only surviving copy is on the device's encrypted flash.

## 9.6 Audit Trail

Every provisioned device is logged with: device public key, certificate serial number, firmware hash, timestamp, and the operator who ran the provisioning. This log serves as the chain of custody for all devices in the network.

## 10. Coordinator Deployment Security

### 10.1 Secrets Management

The coordinator requires several secrets at runtime: its TLS private key, the PostgreSQL database password, the Solana keypair for on-chain transactions, and RPC API keys. None of these are ever stored in source code, plaintext environment variables, or Docker Compose files.

#### Primary: HashiCorp Vault

HashiCorp Vault (self-hosted, open-source edition) serves as the secrets manager. The coordinator authenticates to Vault at startup using AppRole auth (role ID baked into deployment config, secret ID injected at deploy time), pulls its secrets, and holds them only in memory.

#### Minimum Viable Alternative: SOPS + age

If Vault is impractical for a 20-node network, Mozilla's SOPS with age encryption provides secrets-at-rest in the deployment repository. Secrets are encrypted in Git, decrypted during deployment by the CI/CD pipeline, and mounted as files into the Docker container at runtime.

### 10.2 Solana Keypair Protection

The coordinator's Solana keypair controls on-chain transaction authority. This is a dedicated hot wallet holding only the minimum SOL balance needed for transaction fees. The on-chain program's upgrade authority is kept on a separate multisig to prevent program modification even if the coordinator wallet is compromised.

## 11. Solana Program Security

### 11.1 Upgrade Authority: Multisig

After mainnet deployment, the program's upgrade authority is transferred to a 2-of-3 multisig using Squads Protocol. The three keys are: primary operator, co-founder, and a hardware wallet stored in a secure location. Any program upgrade requires two of three signatures.

### 11.2 Audit

Before mainnet deployment, the program must be audited. At minimum, use sec3 (formerly Soteria) for automated scanning. A manual audit from a firm such as OtterSec, Neodyme, or Halborn is strongly recommended for a randomness protocol where financial value is at stake.

### 11.3 Verification

After audit and deployment, the program source code is published and verified using Anchor's anchor verify command, allowing anyone to confirm the on-chain bytecode matches the published source.

## 12. Repository & CI/CD Security

### 12.1 Branch Protection

- No direct pushes to main — all changes through pull requests
- Minimum one code review required per PR
- Signed commits enforced (GPG or SSH signing) to verify authorship

### 12.2 CI/CD Pipeline

- Firmware builds must be reproducible (same source always produces same binary)
- Run cargo clippy and cargo audit on every coordinator PR
- Run Anchor tests and Trident fuzz tests on every smart contract PR
- No secrets stored in the application code repository

### 12.3 Dependency Supply Chain

- Pin exact versions of all dependencies
- Cargo.lock always committed for Rust projects
- ESP-IDF pinned to a specific release tag
- Periodic dependency audits with cargo audit
- Monitor ESP-IDF security advisories

## 13. Server & Network Hardening

### 13.1 Firewall

The coordinator VPS runs a strict firewall (ufw or nftables). Only four ports are open: the mTLS WebSocket port (for nodes), the public API port (for developers querying randomness), SSH on a non-standard port, and Prometheus metrics on localhost only. All other traffic is blocked.

### 13.2 SSH

- Key-based authentication only (password auth disabled)
- Root login disabled
- fail2ban for brute force protection
- SSH access restricted to specific IP addresses where possible

### 13.3 Container Isolation

The coordinator process runs as a non-root user inside Docker with a read-only filesystem. No capability escalations. The container has no access to the host network namespace beyond its exposed ports.

### 13.4 Database Hardening

- PostgreSQL listens on localhost only
- Strong random password, rotated periodically
- TLS enabled for database connections
- Regular automated backups, encrypted, stored off-site

## 13.5 OS Maintenance

Unattended-upgrades are enabled for automatic security patches at the OS level. This is the one component where automatic updates are appropriate.

## 14. Monitoring & Incident Detection

### 14.1 Alerting Rules

- Unexpected node disconnections (potential network attack or device compromise)
- Coordinator restart or crash
- Unusual patterns in randomness requests (sudden spikes may indicate exploitation attempts)
- Failed mTLS handshakes (potential probing with invalid certificates)
- Solana transaction failures
- Database connection issues

### 14.2 Audit Logs

All coordinator actions are logged: which nodes were selected for each request, commit/reveal timing, finalization results. These logs are stored separately from the coordinator (shipped to a log aggregation service or separate server) to prevent tampering if the coordinator is compromised.

## 15. Disaster Recovery

### 15.1 Coordinator Failure

Nodes retry connections automatically. A hot standby coordinator or documented rapid-deployment procedure ensures recovery within one hour. Coordinator state (node registry, active requests) is preserved in PostgreSQL backups.

### 15.2 Device Key Compromised

Deregister the device's public key on-chain via a `deregister_device` instruction. The coordinator blacklists the key. The device is permanently retired.

### 15.3 Intermediate CA Compromised

Sign a new intermediate certificate using the root CA. Reissue the coordinator certificate. Devices with certificates from the compromised intermediate become untrusted and must be reprovisioned. This is the most operationally disruptive scenario short of root CA compromise.

### 15.4 Root CA Compromised

This is a total-loss scenario for the existing network. The entire PKI must be rebuilt and all devices reprovisioned from scratch. This is why root CA key protection is treated as the highest security priority.

### 15.5 Coordinator Wallet Drained

The hot wallet holds only minimal SOL. It is topped up regularly from a separate treasury wallet controlled by the multisig. Loss is bounded by design.



## 16. Secrets Inventory

Every secret in the system has a defined location, a protection mechanism, and a compromise response plan.

| Secret                     | Location                                              | Protection                                               |
|----------------------------|-------------------------------------------------------|----------------------------------------------------------|
| Root CA Private Key        | Two encrypted USB drives, separate physical locations | LUKS + Shamir's Secret Sharing on passphrase             |
| Intermediate CA Key        | Provisioning station disk                             | LUKS full disk encryption, machine powered off when idle |
| Device Private Keys        | ESP32 encrypted flash (per device)                    | Flash encryption + secure boot eFuses burned             |
| Coordinator TLS Key        | HashiCorp Vault or SOPS                               | Injected at runtime, held in memory only                 |
| Coordinator Solana Keypair | HashiCorp Vault or SOPS                               | Hot wallet with minimal balance                          |
| Program Upgrade Authority  | Squads multisig (2-of-3)                              | Hardware wallets for signers                             |
| Database Credentials       | HashiCorp Vault or SOPS                               | Injected at runtime                                      |
| RPC API Keys               | HashiCorp Vault or SOPS                               | Injected at runtime                                      |

*If the answer to “where is this secret and what happens if it leaks?” cannot be stated clearly for every credential in the system, there is a security gap that must be resolved before deployment.*